iEvaluate Batch Scanning System: Handover Guide

Contents

1. What this system is
2. The big picture
3. How one form travels
4. Each part explained
5. What to install
6. Run the website on localhost (fresh Windows PC)
7. Install the folder watcher on a scanning PC
8. Build and install the phone app
9. Maintain n8n and the OMR server
10. Database and migrations
11. Everyday use guide
12. Troubleshooting
13. Known limitations and future work
14. Handover checklist
15. Glossary

For: the Product Owner. Goal: after reading this you can understand, install, run and maintain the system.

How to read it: every technical word is explained the first time it appears. All words are also in the Glossary (section 15).

Facts here come from the code, the project notes and the n8n docs. Where a fact is not known, this guide says so and tells you how to check. Nothing secret is written here. Secrets are shown only by name, with the placeholder `<ask the previous developer>`.

---

1. What this system is

Schools use a paper form called the **SAST** (students rate their instructor on 20 questions by ticking a box). The Student Affairs Office (**SAO**) must turn these paper forms into scores.

This system does that in two ways:

1. **Website path.** Staff scan stacks of forms on a flatbed or document scanner. The pages are sent automatically, read by software, scored, and shown live. Forms that look wrong go to a "Needs review" list for a person to check.
2. **Phone path.** Staff take a photo of one form with the Android app (**iEvaluate**). It goes through the same reading and scoring.

Both paths save to the same cloud database. The phone app also lets instructors, department heads and admins see reports.

Who	Uses what	What they do
SAO staff (`SAO_STAFF`)	Website, scanning PC, phone app	Scan forms, review problem forms
SAO admin (`SAO_ADMIN`)	Website, phone app	Everything staff do, plus activate instructor accounts, manage users, subjects, departments, terms
Instructor	Phone app only	See own scores, remarks, AI suggestions
Department head or dean	Phone app only	See department results, write intervention reports (the app emails the instructor)

Instructors never log in to the website. Only approved accounts can log in (account status must be `approved`).

---

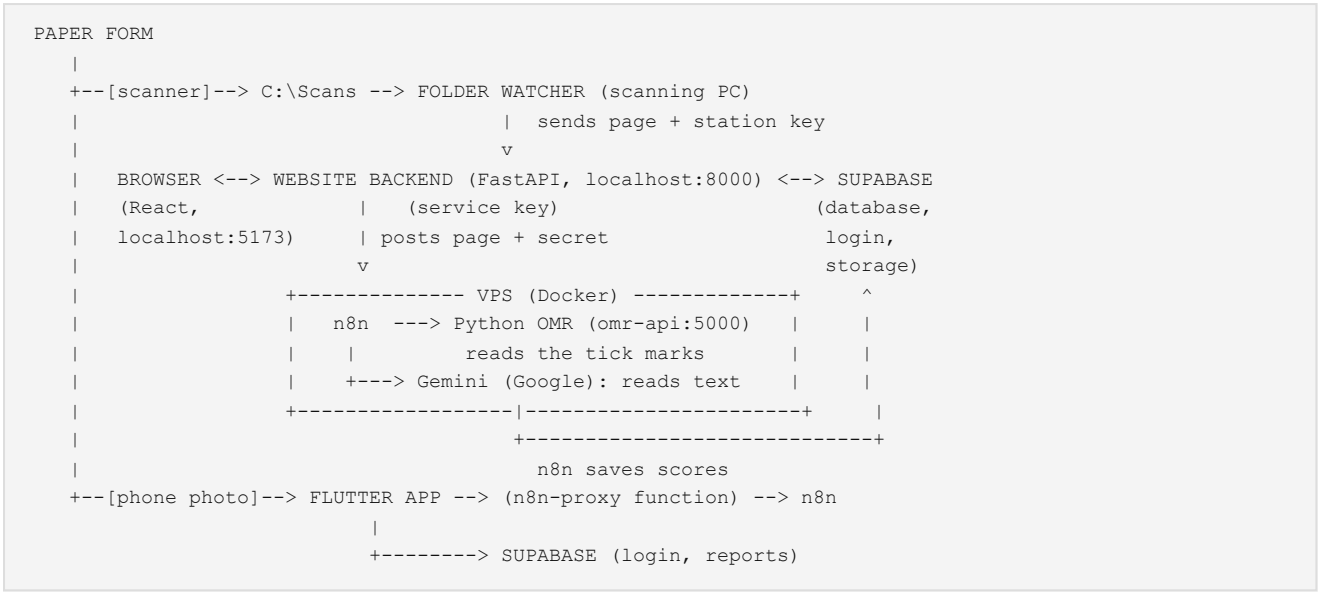
2. The big picture

2.1 Where things run (read this first)

Where	What runs there
Your localhost (a normal PC, not on the internet)	The **website** (React frontend + FastAPI backend). It is NOT deployed anywhere.
A scanning PC	The **folder watcher** (small Python program)
A rented server (VPS, Contabo), inside Docker	**n8n** and the **Python OMR service**
Cloud	**Supabase** (database, login, file storage), **Google Gemini** (AI reading)
Android phones	The **Flutter app** (an APK file)

Important: because the website only runs on localhost, the backend must be running on the PC where staff use the site, and the scanning PC's watcher must be able to reach that backend. See section 7 for what this means.

2.2 Diagram



2.3 Where each part lives

Part	Technology	Runs where	Folder / location
Website frontend	React 18, Vite 5	Localhost, port 5173	`c:\Projects\shisss complee new website\app\frontend`
Website backend	Python, FastAPI, uvicorn	Localhost, port 8000	`...\app\backend`
Folder watcher	Python 3.10+	Each scanning PC	`...\app\backend\watcher`
n8n workflow	n8n (workflow tool)	VPS, Docker	`...\app\n8n\perfect-2.1.json` (the file); tools in `...\app\n8n\tools`
Python OMR/OCR	Python, Flask, OpenCV	VPS, Docker, container `omr-api`	`C:\Projects\OMR PYTHON DEPLOYMENT` (separate repo)
AI	Google Gemini (`gemini-3.5-flash-lite`)	Google cloud, called by n8n	n/a
Database	Supabase (Postgres)	Cloud	SQL files in `...\app\migrations`
Phone app	Flutter / Dart	Android phones	`C:\Projects\ievaluateapp_final` (separate repo)
Mark labelling tool	Static web page	Any browser	`...\app\sast-mark-dataset\index.html`
OMR patch notes	Docs and test scripts	n/a	`...\app\omr_patch`

Hosting files exist for a future online deployment (`render.yaml`, `frontend\vercel.json`, `backend\Dockerfile`, `backend\Procfile`). They are NOT in use.

---

3. How one form travels

3.1 Website / scanner path

1. Staff log in to the website and open "Start" to begin a **batch**. A batch = one instructor + one subject + one scanning PC (called a **station**). Only one batch can be open per station.
2. The scanner saves each page into the watched folder (default `C:\Scans`).
3. The **watcher** sees the new file and converts it to a grayscale JPEG (about 200 dpi). It sends it to the backend only while a batch is open. Files older than the batch are moved aside (`skipped\before-batch`).
4. The **backend** checks the page: valid station key, JPEG or PNG, 12 MB or less, not already sent in this batch. It also checks whether the same image was used in another batch ("reused image").
5. The backend straightens the page (upright, portrait) and crops it to short bond (8.5 x 11) or long bond (8.5 x 13) paper. It adds a thin grey border so the paper can be found.
6. The backend posts the page to **n8n** and waits up to 360 seconds.
7. **n8n** sends the image to the **Python OMR** service (reads which box is ticked on each of 20 rows). It also sends it to **Gemini** (reads printed and handwritten text: instructor, subject, semester, remarks, student ID, stamp number; also double-checks the ticks).
8. n8n compares both readings, matches the instructor, subject and student against the database, and decides.
9. Result is one of four answers: - **success**: saved and counted. - **flagged**: put in the review queue with the reason(s). - **rejected**: not a SAST form, or fewer than 5 of 20 answers, or a copied stamp. - **duplicate**: already seen, nothing changes.
10. After a clean save, n8n recalculates the instructor's averages, runs the AI remark analysis, and may send a push alert (low score or critical remark).
11. The live batch page shows counters. Staff open "Needs review" to fix and credit (approve) or discard flagged forms. Approving calls n8n again.
12. Staff close the batch. A forgotten batch is force-closed after 3 hours by default.

3.2 Phone path

1. Staff take a photo in the app's scanner and check the result on the Data Validation screen.
2. The app calls the `n8n-proxy` function in Supabase. It checks the user's login and role, then forwards to n8n (`sast-scan-upload`). The app waits 60 seconds.
3. n8n does the same OMR + Gemini reading. A phone has no batch, so the instructor and subject are read from the form text and matched locally in the database.
4. Clean forms are saved. Flagged forms go to a "failed scans" list in the app for manual fix.

---

4. Each part explained

4.1 Website (React frontend)

Pages (staff only):

Page	Address	Purpose
Login	`/login`	Staff sign in
Home	`/`	Getting-started checklist
Start a batch	`/start`	Pick or create instructor (placeholder if missing), subject, term
Live batch	`/batch/<id>`	Live counters, page list, page viewer (image + score), close batch
Review	`/batch/<id>/review`	"Needs review" forms, import errors, "Couldn't be saved" pages. Edit and credit or discard.
Review batches	`/review`	All batches the person may review
Scanner laptop	`/scanner-setup`	Online/Offline light, download watcher installer, change scan folder
Activation	`/activate`	Admin only. Give a placeholder instructor a real email so they can use the app (email sent through Brevo)
SAST guidelines	`/sast-guidelines`	Do's and don'ts for filling the form. Shown once per semester
Help	`/help`	"Something went wrong?" table

Key files: `frontend\src\App.jsx` (routes), `api.js` (calls to backend), `messages.js` (all wording, including review reasons). The browser talks only to the backend, never to Supabase or n8n. Login is kept in the browser session, so closing the browser logs you out. There are no frontend tests.

4.2 Folder watcher (version 2.5.0)

A small Python program on the scanning PC. It watches a folder, converts files (`.jpg .jpeg .png .tif .tiff .pdf`, multi-page PDFs are split) and sends them.

- Proves who it is with a **station key** (a password for that PC).
- Sends a "heartbeat" every 15 seconds. The website light turns red about 40 seconds after it stops.
- Sent files go to `<folder>\processed\<date>\`. Pre-batch files go to `skipped\before-batch\`. Unusable files go to `skipped\`.
- Only one copy can run at a time.
- Network folders are refused.
- Since 2.5.0 it does NOT start by itself at log-on. Staff double-click "Start Scan Watcher" before scanning, and "Stop Scan Watcher" after.

4.3 Backend (FastAPI)

The "middleman". It checks logins, owns batches and stations, receives pages, prepares images, calls n8n, and stores page viewing copies.

- Uses the Supabase **service key** (all-powerful secret). It must never be in the browser or in git.
- Health check: `http://localhost:8000/health` returns `{"status":"ok"}`.
- Routers (in `backend\app\routers`): auth, stations, instructors, subjects, departments, batches, batch_pages, scans, reviews, watcher_download.
- Page viewing copies (up to 1800 px) are kept in the private storage bucket `scan-pages` for 90 days.
- Limits: 12 MB per page, 40 megapixels. A page is tried up to 5 times (`MAX_PIPELINE_ATTEMPTS`) on errors, then marked failed. A timeout or network error is retried without limit, so a dead n8n leaves pages "pending".

Scoring and review rules:

Rule	What happens
Only a tick or slash counts	X, shading, circle, dot = treated as blank (not an error)
Blank is empty, not zero	Blank answers are left out of averages (migration 013)
Fewer than 5 of 20 answered	Auto-discarded, never counted. Staff cannot approve it (migration 014)
Python and Gemini disagree a lot	Flagged `omr_agreement_below_30`
Stamp number	Each form carries a numbering-machine stamp. Unique per term. A copy is auto-discarded `duplicate_stamp`. Missing or unreadable stamp goes to review (migration 015)
Reused image	Same image in another batch is flagged `reused_image`
Blank instructor, subject or student ID	Flagged
About 1 in 20 clean pages	Randomly sent to review as a spot check
Approve safety	Cannot overwrite another form of the same student without confirming
Sexual words in remarks	Masked with asterisks (insults are not masked)

Who sees what: admins see everything. Staff see their own batches and the batch of the laptop they are at.

4.4 n8n (the workflow "brain")

n8n is a tool where you connect steps ("nodes") on a canvas. Our workflow is `n8n\perfect-2.1.json` ("perfect 2.1", 191 nodes, live id `vOMBjDtz15QB4k4u`).

Webhook (web address that starts it)	Called by	Purpose
`sast-batch-scan`	Website backend (needs `X-Webhook-Secret` header)	One scanned page
`sast-scan-upload`	Phone app	One phone photo
`sast-manual-correction`	Website Review page, phone app	Staff fixes a flagged scan and credits it
`import-error-correction`	Phone app, backend	Fix a Google Sheet import row
`sast-link-upload`	Phone app	Import a Google Sheet of results
`subject-bulk-import`	Phone app	Bulk create subjects
`health-check`	Phone app	Returns "online"

Notes: - Alerts are push notifications (Firebase). n8n sends no emails. Instructor activation email comes from the backend (Brevo). Intervention emails come from a Supabase function used by the phone app. - Do NOT rearrange the canvas or set an Error Workflow in settings. Order of branches matters. - Node names contain typos (`subjetc`, `departmeht`). Keep them; code refers to nodes by name. - n8n source pieces live in `n8n\tools\website_src\`. See `n8n\WEBSITE_BRANCH.md` (main technical doc).

4.5 Python OMR/OCR service

Code: `C:\Projects\OMR PYTHON DEPLOYMENT` (own git repo). **OMR** = reading ticked boxes. **OCR** = reading text. This service is classic computer vision, not an AI service.

What it does for each image: 1. Finds the paper and the rating table (20 rows x 5 columns, two sections), flattens it. 2. Decides which column is ticked in each row (only ticks count). 3. Finds and reads the stamp number with its own small digit reader. Gemini's read must match it digit by digit, otherwise the form goes to review.

Header text OCR (EasyOCR) is switched OFF (`OCR_MODE=off`) because it was about 0% accurate on handwriting. Gemini reads the header instead. n8n ignores Python text.

Endpoints:

Endpoint	Purpose
`POST /process`	Form image in, ratings + stamp candidates + debug info out
`POST /process-crop`	Reads one hand-drawn crop (phone app Data Validation)

Endpoint	Purpose
`GET /health`	Returns status

Main files: `api.py`, `utlis.py`, `grid_detect.py`, `grid_config.py`, `mark_detect.py`, `stamp_find.py`, `stamp_read.py` (+ weights `stamp_read_weights.pt`, `stamp_templates.npz`), `Dockerfile`, `requirements.txt`.

Switches (environment variables): `OCR_MODE`, `STAMP_FINDER`, `STAMP_READER`, `OMR_MAX_LONG_SIDE`, `OMR_CAP_LONG_SIDE`.

Accuracy (small test sets): about 97% of rows on tuning sets; new table print 37% to 98.8% after the 2026-10-08 fix. It is weak when the photo shows only part of the page or is rotated. In that case it silently guesses a fixed grid (flag `omr_fallback_grid`). This is future work (section 13).

4.6 Gemini AI and privacy rules

- Model: Google Gemini `gemini-3.5-flash-lite`, called directly from n8n (not through OpenRouter).
- What Gemini gets: the form image (student ID and signature are visible in it), the list of known SUBJECTS, and instructions.
- What Gemini never gets: **instructor names**. This is a school rule. Instructors are matched locally in code against the database. The build and the tests check this. Trade-off: badly garbled names go to staff review.
- Remark analysis (themes, word cloud, tone, summary) sends the remark text to Gemini.
- **Privacy warning.** The capstone paper claims OpenRouter, "anonymization" and student-ID hashing. The code does none of these. Student IDs are stored in plain text, and Google is not named as a data processor in the app's privacy notice. The paper or the code must be corrected (section 14).
- Free Gemini keys hit a request limit (error 429). The key is shared with production n8n. Whether the key is on a paid tier is unknown (see section 14).

4.7 Supabase database

Supabase is a hosted database (Postgres) with login and file storage. The website and phone app share one Supabase project. The project URL is hard-coded in 8 n8n nodes.

Main tables: `user_info`, `Sao_users`, `stations`, `scan_batches`, `scan_batch_items`, `scan_unsaved_pages`, `failed_scan_queue`, `import_errors`, `sast_all_raw_data_survey` (the raw scores; key is subject + term + instructor + student), `management_results`, `performance_results`, `overall_total_survey`, `overall_question_means`, `student_remarks`, `instructor_wordcloud`, `instructor_ai_suggestions`, `subjects`, `academic_terms`, `system_settings`, `sheet_import_jobs`, `audit_logs`, and more (the phone app lists about 30 tables).

Key functions: `recompute_sast(instructor, term)` (rebuilds averages), `save_website_raw_row`, `sast_stamp_duplicate_of`.

Storage: private bucket `scan-pages` (page viewing copies).

Supabase **edge functions** (small server programs inside Supabase, in the Flutter repo): `n8n-proxy`, `send-intervention-email`, `update-system-settings`, and admin ones (`admin-create-user`, `admin-accept-user`, `admin-reject-pending`, `admin-update-role`, `admin-activate-account`, `delete-user`, `send-admin-code`).

Security: row-level rules (**RLS**) limit who can read each table. Migration 007 fixed self-signup privilege escalation.

4.8 Flutter phone app (iEvaluate)

Repo: `C:\Projects\ievaluateapp_final`. Version 1.0.0+1. App id `com.example.ievaluateapp_final` (a default name; change before any store release).

Role in app	Database role	Screens
Gatherer (SAO staff)	`SAO_STAFF`	Dashboard, camera scanner, data validation, sync queue, failed scans, Google Sheet import, settings
Instructor	FULL-TIME / PART-TIME / INSTRUCTOR	Dashboard, subjects, student feedback, past terms, PDF report
Dept head / dean	DEPARTMENT-HEAD / DEAN	Overview, faculty roster, subject analytics, intervention reports
SAO admin	`SAO_ADMIN`	User, personnel, subject, department management, system audit, import errors, live metrics, settings

The app talks to Supabase directly, and to n8n only through `n8n-proxy`. Push messages use Firebase Cloud Messaging (`google-services.json` is present).

---

5. What to install

Needed for	Software	Version	Notes
Website (backend)	Python	3.11 or 3.12	Docker image uses 3.11. No exact local version is recorded. Check with `python --version`
Website (frontend)	Node.js + npm	Not recorded. Use 20 LTS	Vite 5 / React 18 need Node 18 or newer (assumption). Check with `node --version`
All	Git	any recent	
Watcher PC	Python	3.10 or newer (3.10 to 3.14 work)	The installer can install it for you (winget)
Python OMR (local test only)	Python	3.10	Matches the production container. A 3.14 install has been used for evaluations
n8n + OMR on VPS	Docker + Docker Compose	on the VPS	Already installed there

Needed for	Software	Version	Notes
Phone app	Flutter	3.41.6 stable (Dart 3.11.4)	Used by the previous developer
Phone app	Android Studio (Android SDK)	any recent	
Phone app	Supabase CLI	any recent	Only to deploy edge functions
n8n tools (optional)	Python 3 and Node	any	For build and test scripts

Accounts needed: Supabase project, Google (Gemini API key; Google Cloud service accounts for Firebase and Sheets), Brevo (email), GitHub repos, VPS provider (Contabo), DuckDNS (name of the n8n address).

---

6. Run the website on localhost (fresh Windows PC)

1. Install Python, Node.js and Git (section 5). Open a new terminal afterwards.
2. Get the code: clone or copy the `app` folder. Open a terminal in it.
3. Make sure the database is ready (section 10). Have the Supabase project URL and the **service key** ready.
4. Create the backend settings file: - Copy `backend\.env.example` to `backend\.env`. - Fill in `SUPABASE_URL` and `SUPABASE_SERVICE_KEY` (secret). - For real scoring also set `N8N_BATCH_WEBHOOK_URL` and `N8N_WEBHOOK_SECRET` (secret; must equal the value in the n8n credential). The backend refuses to start if the URL is set without the secret. - For a demo without n8n: leave the URL empty and set `SIMULATE_SCANS=true`. You get fake results. - `.env` is private. Never commit or email it.
5. Install and start the backend: `cd backend python -m venv venv venv\Scripts\python -m pip install -r requirements.txt venv\Scripts\python -m uvicorn app.main:app --reload --port 8000`
6. Check `http://localhost:8000/health`. You should see `{"status":"ok"}`.
7. Open a second terminal. Install and start the frontend: `cd frontend npm install npm run dev`
8. Open `http://localhost:5173` in a browser. The frontend finds the backend at `http://localhost:8000`. To change it, set `VITE_API_BASE` in the frontend environment.
9. Log in. You need an approved account: a row in `user_info` with `account_status = approved` and a row in `Sao_users` with role `SAO_STAFF` or `SAO_ADMIN`. A new station is registered the first time you use the "Scanner laptop" page.

Backend settings (names only; defaults are in `backend\.env.example`):

Name	Meaning
`SUPABASE_URL`, `SUPABASE_SERVICE_KEY`	Database address and secret key (required)

Name	Meaning
`N8N_BATCH_WEBHOOK_URL`, `N8N_WEBHOOK_SECRET`	n8n address and shared secret
`N8N_CORRECTION_WEBHOOK_URL`, `N8N_IMPORT_ERROR_WEBHOOK_URL`	Optional. Derived from the batch URL if empty
`SIMULATE_SCANS`	Fake results, no n8n
`CORS_ORIGINS`, `CORS_ALLOW_LAN`	Which web addresses may call the backend
`ABANDONED_BATCH_HOURS`	Force-close forgotten batches (default 3)
`N8N_TIMEOUT_SECONDS`, `N8N_CORRECTION_TIMEOUT_SECONDS`, `MAX_PIPELINE_ATTEMPTS`	Timeouts (360, 180) and retries (5)
`MAX_UPLOAD_MB`, `INGEST_MAX_CONCURRENCY`, `PREDATE_GRACE_SECONDS`	Upload limits
`WATCHER_ONLINE_SECONDS`, `WATCHER_API_BASE`, `PUBLIC_API_BASE`	Watcher light timing and the address given to watchers
`PAD_FOR_OMR`, `PAD_PERCENT`, `CROP_LONG_BOND`	Image preparation for OMR
`SCAN_IMAGE_BUCKET`, `SCAN_IMAGE_RETENTION_DAYS`, `SCAN_IMAGE_MAX_SIDE`, `SCAN_IMAGE_JPEG_QUALITY`	Page viewing copies
`BREVO_API_KEY` (secret), `BREVO_SENDER_EMAIL`, `BREVO_SENDER_NAME`	Activation emails
`TRUST_PROXY_HEADERS`, `FORWARDED_ALLOW_IPS`	Only when behind a proxy

Frontend settings: `VITE_API_BASE`, `VITE_SHEET_SERVICE_ACCOUNT_EMAIL` (shown on screen so staff know which email to share a Google Sheet with).

Run the tests

```
cd backend
venv\Scripts\python -m pip install -r requirements-dev.txt
venv\Scripts\python -m pytest -q
```

Tests do not write to the database. Known: one test (`test_orientation` real samples) failed before recent changes and still may. Other tests:

Command (from `n8n\`)	Checks
`python tools/validate_workflow.py`	Workflow structure and rules (optional `pip install pglast`)
`node tools/test_website_code.js`	The workflow's code nodes

Command (from `n8n\`)	Checks
`node tools/test_website_sql.js`	Its SQL. Needs a throwaway local Postgres. Refuses non-local hosts

Phone app tests: `flutter test`.

---

7. Install the folder watcher on a scanning PC

Because the website runs only on localhost, the watcher can reach it only if the backend is reachable from the scanning PC. Easiest: run the backend and website on the scanning PC itself, or on a PC in the same network with `WATCHER_API_BASE` and `PUBLIC_API_BASE` set to its address. The right address for your setup is not recorded; confirm with the previous developer.

Steps (done by staff, no technical skill needed):

1. Log in to the website on that PC. Open "Scanner setup".
2. Choose or create the station (the laptop's name).
3. Click "Download installer". You get `ScanWatcher-<station>.zip`. Each download makes a NEW station key, so the old PC using that station stops working.
4. Right-click the zip, choose "Extract All".
5. Open the extracted `ScanWatcher` folder. Double-click `INSTALL.bat`.
6. Wait for the green "ALL DONE" window. The installer finds Python (or installs it), installs what it needs, copies the program to `%LOCALAPPDATA%\ScanWatcher`, creates `C:\Scans`, and adds "Start Scan Watcher" and "Stop Scan Watcher" shortcuts to the desktop and Start menu.
7. In the scanner software, set the save folder to `C:\Scans`.
8. Before each scanning session, double-click "Start Scan Watcher". The website light should turn green (Online).

Notes: - If the installer window closes at once, the zip was not extracted. - To remove it: `UNINSTALL.bat`. To check status: `CHECK.bat`. - IT can make it start at log-on with `install_task.bat` (undo with `uninstall_task.bat`). Running `INSTALL.bat` again turns auto-start off. - Laptops installed before 2.5.0 have no shortcuts and must reinstall. The website shows a note when the version is older than 2.5.0. - The zip is `backend\app\static\watcher-package.zip`. After ANY watcher edit rebuild it: `cd backend` then `venv\Scripts\python watcher\build_package.py`. Never put a `watcher_config.json` inside the zip. - Files in `backend\watcher`: `INSTALL.bat`, `UNINSTALL.bat`, `CHECK.bat`, `STOP.bat`, `shortcuts.bat`, `start_watcher.bat`, `watcher_process.bat`, `find_python.bat`, `setup_helper.py`, `folder_watcher.py`, `build_package.py`, plus guides `HOW TO INSTALL.txt` (end users), `TECHNICAL NOTES.txt` (IT), `INSTALL_STEPS.md` (wording shown on the website; keep in sync).

---

8. Build and install the phone app

Do this on the developer PC, in `C:\Projects\ievaluateapp_final`.

1. Install Flutter 3.41.6 and Android Studio. Run `flutter doctor` and fix what it reports.
2. Run `flutter pub get`.
3. Create `.env.json` in the project root (never commit it). Keys, names only: `SUPABASE_URL`, `SUPABASE_ANON_KEY`, `N8N_BASE_URL`, `USE_N8N_TEST`, `N8N_WEBHOOK_HEALTH`, `N8N_WEBHOOK_SCAN_UPLOAD`, `N8N_WEBHOOK_LINK_UPLOAD`, `N8N_WEBHOOK_CROP_OCR`, `N8N_WEBHOOK_MANUAL_CORRECTION`, `N8N_WEBHOOK_SUBJECT_BULK_IMPORT`, `N8N_WEBHOOK_IMPORT_ERROR_CORRECTION`, `PYTHON_PROCESS_CROP`. Optional: `USE_DIRECT_N8N`, `SHEET_SERVICE_ACCOUNT_EMAIL`. Values: `<ask the previous developer>`.
4. Test run on a connected phone: `flutter run --dart-define-from-file=.env.json`.
5. Build the release file: `flutter build apk --release --dart-define-from-file=.env.json`. Without the define file, login fails (URL and key are empty).
6. The file is `build\app\outputs\flutter-apk\app-release.apk`.
7. Install on a phone: turn on USB debugging and run `adb install -r build\app\outputs\flutter-apk\app-release.apk`, or copy the APK to the phone and open it.

Important: - **Signing key.** Release builds use `android\app\keystore\release.jks`. Back it up. If you lose it, users must uninstall before they can update. If the file is missing, the build silently uses a debug key. Environment overrides: `KEYSTORE_FILE`, `KEYSTORE_PASSWORD`, `KEY_ALIAS`, `KEY_PASSWORD`. The alias and password have default values written inside `build.gradle.kts`; remove and rotate them. - Debug and release builds have different signatures. Switching gives `INSTALL_FAILED_UPDATE_INCOMPATIBLE`. Fix: `adb uninstall com.example.ievaluateapp_final`. This deletes scans not yet synced. - Judge speed on release builds only. Debug builds lag. - The Android `INTERNET` permission is in `AndroidManifest.xml` (without it, release login broke). - Deploy a Supabase function: `supabase functions deploy <name>`.

---

9. Maintain n8n and the OMR server

I did not log in to the server. Values below come from project notes. Verify them on the server.

9.1 The server

- Provider: Contabo VPS. Stack folder: `/root/ievaluate-stack/` (Docker Compose). Login details: `<ask the previous developer>`.
- n8n address: a DuckDNS name served over HTTPS by Caddy (`docker-compose.override.yml` + `Caddyfile`; backup `docker-compose.yml.bak`). Exact name: `<ask the previous developer>`.
- Containers: n8n, `omr-api` (Python OMR, port 5000 inside the Docker network only; n8n reaches it as `http://omr-api:5000/process`), and Caddy.
- Not yet done (from notes): close the old open port 5678, set the Supabase secret `N8N_BASE_URL` for `n8n-proxy`, and reboot the server (it asked for a restart).

9.2 Import the workflow into n8n

1. Open n8n. Open the workflow "perfect 2.1".
2. Top-right menu, "Import from File", choose `n8n\perfect-2.1.json`. This replaces the canvas and keeps the web addresses.
3. Any node with a red credential marker: pick the right credential from its dropdown.
4. Save. Switch the workflow to Active. (The doc says the file imports inactive. The file in the repo has `active: true`. Check after import.)
5. Webhook addresses starting `/webhook/` only work while Active.
6. If the default 16 MB request size is too small, set `N8N_PAYLOAD_SIZE_MAX`. Pages are about 1 MB.

9.3 Credentials to create in n8n (names must match exactly)

Name	Type	Value
`iEvaluate website webhook secret`	Header Auth, header `X-Webhook-Secret`	Same as backend `N8N_WEBHOOK_SECRET`
`Supabase iEvaluate (secret key)`	Supabase API (host = project URL)	Supabase service key
`Postgres account`	Postgres	Supabase database connection
`Google Gemini(PaLM) Api account`	Google Gemini (PaLM) API	Gemini key
`firebase pushnotification`	Google API (service account)	For push messages, Firebase project `ievaluate-pushnotification`
`Google Service Account account 2`	Google API (service account)	For Google Sheets

All values: `<ask the previous developer>`. After a Supabase key change, update both n8n and the backend `.env`.

9.4 Restart or update the OMR service

1. Log in to the VPS. Go to `/root/ievaluate-stack/`.
2. Restart: `docker compose restart omr-api`.
3. Update: copy the changed files into the stack's `omr-api` folder, then `docker compose up -d --build omr-api`.
4. Check: `curl http://localhost:5000/health` from inside the Docker network (it can refuse for 30 to 60 seconds after a restart).
5. After every update, compare file checksums of `api.py`, `mark_detect.py`, `grid_detect.py`, `utlis.py`, `grid_config.py` and the stamp files with your local copies. A past rollback left old files running.
6. Rollback aids: backups under `/root/omr_backup/<date>`, `/root/omr-api.bak-<date>` and the Docker image tag `omr-api:rollback`.

7. Phone photos: the size limits `OMR_MAX_LONG_SIDE` and `OMR_CAP_LONG_SIDE` should be 4200 and 3264 (the code default in the local `api.py` still shows 3000 and 1920; the notes say the VPS runs the higher values). Confirm on the VPS.

9.5 Change the workflow

Never edit the JSON by hand if you can avoid it. 1. Edit the pieces in `n8n\tools\website_src\`. 2. From `n8n\`: `python tools/build_website_branch.py --live "<exported live workflow>.json" ../perfect-2.1.json`. This patches a live export (the normal way). 3. Run the tests (section 6). 4. Import into n8n (9.2).

Note: the build script was written to read input files from the previous developer's Downloads folder (`perfect 101 (4).json`, `(5).json`). Those may be gone; always start from a fresh export of the live workflow.

---

10. Database and migrations

Migration = a SQL file that changes the database. There is NO automatic runner and no "applied" log. Each file is run by hand in the Supabase SQL Editor. Read each one first. Most can be safely re-run.

How to check what is applied: ask a developer to run Supabase `list_migrations`, or inspect the objects (for example, look for the table or column a migration creates, as in the "Check" column).

File	What it does	Check
001_batch_scanning	`stations`, `scan_batches` tables	Tables exist
002_batch_hardening	Station key hash, batch counters, `scan_batch_items`, tighter rules. **Required.** Without it the backend returns `schema_outdated`	Table `scan_batch_items`
003_repair_import_error_scan_images	One-off repair, optional	n/a
004_mask_existing_profanity	One-off clean-up. Back up first, cannot be undone	n/a
004_watcher_folder_status	`watcher_*` columns on `stations`	Column on `stations`
005_scan_review_queue	Review versioning. Without it review actions return `migration_required`	Column `review_version`
006_ocr_correction_log	View `ocr_correction_log`	View exists
007_audit_queries	Read-only. Run FIRST	n/a

File	What it does	Check
007_security_hardening	Fixes self-signup privilege escalation, RLS	Notes say applied 2026-09-30
008_scan_unsaved_pages	Table for "Couldn't be saved" pages	Notes say applied
009_stations_watcher_skipped_files	Extra watcher columns	Column `stations.watcher_skipped_files`
010_scan_page_images	Page viewer. Needs private bucket `scan-pages`	Bucket exists
011_instructor_name_matching	Duplicate-instructor detection	Function `find_duplicate_instructor`
012_scan_items_sha_index	Index for the reused-image check	Index on `file_sha256`
013_sast_null_aware_means	Blank = NULL; `recompute_sast`	Function `recompute_sast`. Notes (2026-10-08) say live
014_sast_min_answered	Fewer than 5 of 20 not counted	Notes (2026-10-16) say applied
015_stamp_no	Stamp columns, unique per term, `save_website_raw_row`	Column `stamp_no`
015b_delete_dummy_scan_data	**DESTRUCTIVE.** Deletes test scans (backs up to `backup_*` tables). Run only on purpose	n/a
016_wordcloud_words_backfill	One-off	n/a
017_null_placeholder_comments	Optional clean-up	n/a
018_sheet_import_jobs	Table for Google Sheet import jobs	Table exists

Two files share number 004 and two share 007; run them by name. The README list is out of date (it omits 006 to 018). The phone repo has its own migrations in `supabase\migrations` (latest `20240130000033`).

Extra database checks: - The unique index on `sast_all_raw_data_survey` must be a plain unique index, not `NULLS NOT DISTINCT`. Otherwise forms with a blank student ID after the first are flagged `duplicate_student_id`. SQL to check is in `n8n\WEBSITE_BRANCH.md`, section "Unique key". Not yet checked on the live database. - Needs the `pg_trgm` extension (name similarity).

Create the first admin: add the person in Supabase Auth, then a `user_info` row (`account_status = approved`) and a `Sao_users` row (role `SAO_ADMIN`).

---

11. Everyday use guide

Staff, each scanning session 1. Double-click "Start Scan Watcher" on the scanning PC. 2. Log in to the website. Check "Scanner laptop": light is green. 3. Click "Start". Choose the instructor, subject and term. Start the batch. 4. Scan the forms into `C:\Scans`. Watch the counters on the live page. 5. When the pile is done, close the batch. Double-click "Stop Scan Watcher". 6. Open "Needs a look" (Review). For each form: compare the picture with the typed values, fix, then Credit or Discard. Stamp problems: type the stamp number, or choose "Form has no stamp".

Admin - Activate an instructor: open "Activation". Type the real email twice. The instructor gets an email. - Manage users, subjects, departments and terms in the phone app. - Each scanning laptop is a "station". Regenerating its key stops the old PC.

Reading the flags (full list in `frontend\src\messages.js`): `identity_mismatch`, `instructor_missing`, `subject_missing`, `student_id_missing`, `duplicate_student_id`, `reused_image`, `orientation_uncertain`, `low_confidence`, `multiple_marks`, `blank_answers`, `grid_not_detected`, `omr_unavailable`, `omr_fallback_grid`, `omr_agreement_below_30`, `layout_mismatch`, `stamp_missing`, `stamp_unreadable`, `multiple_stamps`, `stamp_unconfirmed`, `duplicate_stamp`.

Approval states: "Sending..." then "Not sent" after 10 minutes if n8n does not answer.

---

12. Troubleshooting

Problem	Likely cause	Fix
Website light is red (Offline)	Watcher not running	Double-click "Start Scan Watcher". Run `CHECK.bat`
"Folder not found" or "That folder isn't allowed"	Wrong or network folder	Set the folder to `C:\Scans` on the Scanner setup page
Installer window closes at once	Zip not extracted	Right-click the zip, "Extract All", run again
Scans do not appear	Scanner saves elsewhere, or no open batch	Point the scanner at `C:\Scans`. Open a batch first
Old files not sent	They are older than the batch	They are in `skipped\before-batch`. Rescan
Old laptop stopped after another download	Download made a new station key	Reinstall on the right PC
Pages stay "Working on it"	n8n or OMR down or slow (timeouts retry forever)	Check n8n is Active and `omr-api` is running (section 9). Pages continue afterwards

Problem	Likely cause	Fix
Everything goes to review as `stamp_unconfirmed`	OMR service is not sending stamp data	Deploy the current OMR files, check `STAMP_FINDER` and `STAMP_READER` are `on`
Many `omr_fallback_grid` or weak rows on phone photos	Image size cap too low on OMR server	See 9.4 step 7
Backend says `schema_outdated` or `migration_required`	Migration 002 or 005 not applied	Apply it (section 10)
Backend will not start	`N8N_WEBHOOK_URL` set without `N8N_WEBHOOK_SECRET`, or missing Supabase keys	Fix `backend\.env`
n8n answers 401 or 403, backend shows 503	Webhook secret mismatch	Make the n8n credential equal `N8N_WEBHOOK_SECRET`
Flagged pages fail to save (401)	n8n "Supabase iEvaluate (secret key)" has an old key	Paste the current key into that credential
Sheet import fails with 403	Sheet not shared	Share it with the service account email (Viewer)
Phone app login fails on release build	Built without `--dart-define-from-file=.env.json`	Rebuild with it
`INSTALL_FAILED_UPDATE_INCOMPATIBLE`	Debug and release signatures differ	`adb uninstall com.example.ievaluateapp_final`, then install
Gemini 429 errors	Request limit	Wait, or use a paid key
Scores look different after a fix	Stale old totals, or blank rule	Expected after 013: averages are recalculated
Review shows "Already approved / changed" (409)	Someone else edited the row	Reload and look again
Browser logs out	Login is kept for the session only	Log in again
Page shows no preview	Migration 010 or bucket missing	Apply 010, create bucket `scan-pages`

13. Known limitations and future work

Limits today: - The website is localhost only. No online deployment. - Sessions end when the browser closes. Login attempts are rate-limited per IP. - Only a tick or slash counts as an answer. This is a rule-based check, not a trained model. - Table finding is fragile: a photo where the page touches the frame, is cut off, rotated, or shows only the table can fail silently and use a guessed grid. Measured: normal framing 10/10, page touching frame 1/10, cut-off edges 2/10, table-only 0/10, rotated 5

degrees 3/10. - Dots versus tiny real ticks cannot be separated reliably. Pencil shading and circles are read less well. - Stamp numbers of 5 or 6 digits never auto-accept. Auto-accept was 17 of 29 pilot pages with 0 wrong. - Gemini is the weaker reader of ticks, so Python wins when they disagree. Thresholds were not re-measured after the latest rules. - Garbled instructor names go to staff review (no AI help by design). - Re-importing the same Google Sheet adds blank-student rows again. A fix idea (stable `task_id` per row) is not built. - n8n crash during a sheet import is not caught (the app stops waiting after 60 minutes). - Google Sheet import errors show only in the phone app, not on the website. - Website audit items not yet decided: n8n down leaves pages "Working on it" forever (no error mark), re-sent file shows no "Already scanned", forgotten-batch reason is only a tooltip, laptop names are case-sensitive, weak activation-email check, student-ID format not normalised for the overwrite guard. - The backend still sends 0 for a blank in `review_logic.py` (around lines 630 and 653). n8n converts it, but it should be `None`. - No frontend tests. Backend tests do not touch the database. - Phone app tech debt: gatherer screens bypass the main scaffold; default app id; keystore passwords in gradle; README mentions a deleted screen. - Security items still open (phone side): `user_info` policy "Allow anonymous email lookup" lets anonymous users read all profiles (needs an app change); `admin-accept-user` does not check the caller is approved; HTML escaping in `update-system-settings`; `intervention_reports` insert rule needs tightening; one account holds both SAO_STAFF and FULL-TIME roles and should be checked by hand.

Future work (frozen by the owner until after the capstone presentation, meant for a future student or professor): - Better table finding: search at several scales, try all 4 rotations, match the form's own line pattern, send "table not found" to review instead of guessing, learned corner detector. Printed corner markers would be ideal but the school cannot reprint the form. - Data needed: 30 to 50 hard phone photos with corners marked, and 3 to 5 blank 300 dpi forms. Use the labelling tool `sast-mark-dataset\index.html`. Zips go to `C:\Projects\OMR PYTHON DEPLOYMENT\dataset\`. This data holds real names and IDs; keep it out of git. - Check-only mark classifier trained on that dataset (replace the rule-based check). - Header OCR training (printable sample sheets exist in `tools\handwriting_sheets\`), student-ID ink cross-check, stamp reader extra training. - Teacher's list of valid stamp prefixes and lengths (currently a placeholder). - Deploy online (Render + Vercel files exist) and the fixes in the pre-deploy audit. - Phone text and screens for the newer review reasons (`stamp_unconfirmed` and others).

---

14. Handover checklist

14.1 Give the Product Owner

Item	Where / note
Website repo	`c:\Projects\shisss complee new website\app` (git, branch `main`). Many uncommitted changes (see 14.3)
Phone app repo	`C:\Projects\ievaluateapp_final`

Item	Where / note
OMR repo	`C:\Projects\OMR PYTHON DEPLOYMENT` (GitHub `4harvey7/OMR_Python_for_SAST`). Has uncommitted files already running on the VPS
Docs	`README.md`, `SETUP.md`, `n8n\WEBSITE_BRANCH.md`, `n8n\README`, `omr_patch\README.md`, watcher guides, phone `README.md`, `MANUAL.md`, `docs\UAT_*.txt`, website `UAT-iEvaluate-Batch-Scanning-Website.docx`
Accounts	Supabase (owner), Google (Gemini key, Firebase, Sheets service accounts), Brevo, GitHub, Contabo VPS, DuckDNS, Android keystore owner
Server	VPS login and Docker stack access: `<ask the previous developer>`

14.2 Secrets to hand over securely (password manager, never email or chat) and rotate

Names only. Values: `<ask the previous developer>`.

Secret	Used in
`SUPABASE_SERVICE_KEY` (service key)	Backend `.env`, n8n credential
Supabase DB password	n8n `Postgres account`
`SUPABASE_ANON_KEY`	Phone app `.env.json`
`N8N_WEBHOOK_SECRET`	Backend `.env`, n8n credential
n8n admin login and n8n API key	n8n
Gemini API key	n8n credential
Firebase and Google service-account files	n8n credentials, `google-services.json`
`BREVO_API_KEY`	Backend `.env`
Station keys	One per laptop; regenerate when handing over
`release.jks` and its passwords	Phone app signing. Back up
VPS login	Server

Rotate now: 1. The Supabase secret key. An old one was found inside an n8n export. Notes say rotation is still pending (`SETUP.md` step 0). After rotating, update the backend `.env` and the n8n credential `Supabase iEvaluate (secret key)`. 2. The n8n API key (it was shared in chat). 3. The hard-coded keystore alias and passwords in `build.gradle.kts`. 4. Delete old `perfect 101*.json` exports from the previous developer's Downloads folder (they may contain the old key).

14.3 Pending items, one by one (from the project notes)

Notes are dated. Check live state before acting, because some may already be done.

Not committed (uncommitted work in the working tree): 1. Website repo: watcher 2.5.0, review and stamp changes, frontend changes, `n8n\perfect-2.1.json`, `SastGuide.jsx`, tests, `watcher-`

`package.zip` (many files). 2. OMR repo: `api.py`, `utlis.py`, `requirements.txt`, `Dockerfile`, `.dockerignore`, `stamp_find.py`, `stamp_read.py`, weights and templates, `tools\`. Most are already deployed on the VPS. Do not commit `dataset\` or `eval_out\` (real names and IDs). 3. Phone repo: changes to `env.dart`, `failed_scan_detail_screen.dart` (large), `gatherer_settings_view.dart`, `admin_dashboard.dart`, `sao_admin_settings.dart` and others; new `correction_pickers.dart`, `sheet_import_job.dart`, `lib\sast_guide\` and 4 tests. A `.mcp.json` file must not be committed. 4. Branch `reused-image-and-heartbeat` (commit 76784c7) was never pushed, and the website's `main` had later commits (68bcea4, 0906f7b) also not pushed (as of the notes).

Not deployed or not applied: 5. n8n: re-import `n8n\perfect-2.1.json` (reused-image check, no instructor names to Gemini on the phone path, stamp rule, id-based corrections, min-5 rule). Notes say an import happened about 2026-10-08. Whether the newest version is live is unknown. 6. Migrations: 013 and 014 were reported live (2026-10-08 and 10-16). 012, 015, 016, 018 (and 009, 010, 011, 017) were not confirmed. 015b is destructive; run only on purpose. Rule for 015: do not turn the stamp rule on until the OMR stamp files, the n8n stamp step, and the teacher's prefix list are settled. 7. Website backend restart and frontend rebuild after the min-5 and review changes: unverified. 8. Phone app: build a new APK after n8n and migration 013, in that order (migration, then n8n import, then the app). Also confirm app averages match the database after 013 (shared `sast_rating` util was added). 9. Phone text for the new reason codes (`reused_image`, `instructor_missing`, `subject_missing`, `student_id_missing`, `stamp_unconfirmed`). 10. Supabase function `send-intervention-email`: it was open to anyone and injectable. Fix (login check, dean check, `report_id`, HTML escaping, `verify_jwt=true`) is probably committed (commit 231242e). Confirm it is deployed with `supabase functions deploy send-intervention-email` and that the `emailed_at` migration is applied. Follow-ups: tighten `intervention_reports` insert rule, escape HTML in `update-system-settings`. 11. OMR files in the previous developer's scratch folders (not in the repo, not deployed): v2 table finder (frozen), and the dot rule (`OMR_DOT_MIN_FRAC`). Decide with the future team. 12. Watcher 2.5.0: every scanning laptop must re-run `INSTALL.bat`. 13. VPS clean-up: close port 5678, set `N8N_BASE_URL` secret for `n8n-proxy`, reboot the server. 14. Confirm on one phone scan that `omr_input_scale` is 1 and `grid_source` is `structure-fit`. 15. Confirm the unique index on `sast_all_raw_data_survey` is not `NULLS NOT DISTINCT`. 16. Delete the dead file `n8n\tools\website_src\recompute_aggregates.sql` if it still exists. Delete the test placeholder instructor "rodz Licayan" and reset test laptops' folder to `C:\Scans`. 17. Website audit bugs awaiting a decision (section 13). 18. Privacy paperwork (capstone paper vs code): - Add Google / Gemini as a named processor in the signed privacy notice (`lib\core\config\agreements.dart` in the phone repo). - Either build student-ID hashing or remove the claim. - Correct the paper: no OpenRouter, no anonymization, model is `gemini-3.5-flash-lite`. - Check whether the Gemini key is on a paid tier (affects whether Google may train on the data). 19. Teacher's answer needed: valid stamp prefix and length list. 20. Rule for the team: the previous developer never pushed or deployed for the owner. Use the commands in this guide yourself.

14.4 First-week checklist for the new owner

- [ ] Receive repos, accounts and secrets through a password manager
- [ ] Rotate keys in 14.2
- [ ] Run the website on localhost (section 6) and run the tests

- [ ] Log in to the VPS; confirm n8n and `omr-api` are running
- [ ] Check which migrations are applied (section 10)
- [ ] Import the latest workflow and check credentials (section 9)
- [ ] Commit the uncommitted work after review
- [ ] Reinstall the watcher on each scanning PC (section 7)
- [ ] Build and test the phone APK (section 8)
- [ ] Scan 5 test forms end to end; confirm one reaches review and one is credited

---

15. Glossary

Term	Meaning
SAST	The paper evaluation form where students rate an instructor on 20 items
SAO	Student Affairs Office
Batch	One scanning run: one instructor, one subject, one station
Station	A scanning PC registered on the website
Station key	A password for one station, used by the watcher
Watcher	Program on the scanning PC that sends scanned files to the backend
Heartbeat	A "still alive" signal sent every 15 seconds
Backend	The server program that does the work behind the website (FastAPI)
Frontend	The pages you see in the browser (React)
Localhost	The same computer you are using. Not visible on the internet
Webhook	A web address that starts a workflow when something is sent to it
n8n	Tool that runs the step-by-step workflow that reads and saves forms
Workflow / node	The whole flow chart / one box in it
Credential	A saved login or key inside n8n
VPS	A rented server
Docker	Packaging that runs a program in an isolated "container"
Container	One running Docker program (for example `omr-api`)
OMR	Optical mark recognition: reading ticked boxes
OCR	Optical character recognition: reading text
Gemini	Google's AI model that reads the form text
Supabase	Hosted database, login and file storage
Migration	A SQL file that changes the database structure
RLS	Row-level security: rules for who can read or change each row
Edge function	A small program running inside Supabase
Service key	Supabase secret key with full power. Never expose it

Term	Meaning
Anon key	Supabase public key for apps. Safe to bundle, limited by RLS
Stamp	Number printed by a numbering machine on each paper form. Unique per term
Flagged	A form sent to review because something looks wrong
Credit	Approve a flagged form so it counts
Placeholder instructor	An instructor record created before the person has a real login
Short / long bond	Paper sizes: 8.5 x 11 and 8.5 x 13 inches
Dpi	Dots per inch: scan sharpness
APK	The Android app install file
Keystore	The file that signs the Android app. Losing it blocks updates
FCM	Firebase Cloud Messaging: push notifications
Brevo	Email sending service used for activation emails
Caddy	Web server on the VPS that adds HTTPS to n8n
DuckDNS	Free service that gives the VPS a web name
Vite	Tool that runs and builds the frontend
pytest	Python test runner
UAT	User acceptance test
`.env`	A private settings file holding secrets. Never commit it